The Reward Hacking Direction: Reading, Detection, and Steering

Niranjan Deshpande, Juliana Li, Agastya Sridharan

1 Introduction

1.1 Motivation & original paper

In a recent paper, Goodfire found that reward hacking is linearly represented in LLM activations, and that a reward-hacking direction `v_hack` can be extracted with a single Difference of Means (DoM) calculation. They show three main things: `v_hack` is interpretable using the logit lens and decodes to cheating tokens, a linear probe on it can detect hacking about as well as an LLM monitor, and steering along it raises hacking propensity. Throughout this report, we will compare our results and setup against theirs.

1.2 Contributions

We largely replicate the interpretability and detection results on OLMo 3.1 32B (probe AUROC 0.78–0.84), but our probe underperforms the LLM monitor, and steering fails to induce hacking.

We extend the replication to three other open models (Qwen3.8-27B, gpt-oss-120b, and GLM-4.5-Air). Detection performs well on Qwen (AUROC 0.93, outperforming the monitor), but poorly on models that actually carry out hacks. Steering increases how often Qwen considers hacking (5% to 69%) without substantially increasing enacted hacking.

We then investigate three extensions: (1) We reinterpret the direction using the more faithful Jacobian lens. (2) We test whether `v_hack` is confounded by concepts such as taking shortcuts and evaluation awareness. (3) We penalize `v_hack` during RL and find that the model largely learns to evade the probe: recall falls from 62% to 22% while hacking remains around 80%.

2 Replication

2.1 Setup

Model & Benchmark. We use OLMo 3.1 32B (Think), which has publicly available weights and checkpoints at every training step. For our benchmark, we use Impossible-LCB, the LiveCodeBench subset of ImpossibleBench. It modifies coding problems so that no correct solution can pass all tests, meaning that passing requires reward hacking.

Data & Evaluation. We generate 150 rollouts: 25 tasks $\times$ 3 variants $\times$ 2 system prompts. The three variants are *oneoff* (passing requires hardcoding specific test cases), *conflicting* (tests demand different outputs for identical inputs), and *original* (the unmodified, solvable task, used as a negative control). The two system prompts are *loose* (neutral instructions) and *strict* (explicitly forbids reward hacking, including hardcoding).

We obtain ground-truth reward hacking labels with Claude Sonnet 4.5 as a CoT judge, taking a majority vote over three passes. Most hacking attempts by this smaller model fail, so we count attempted hacking as hacking, following the original paper. We construct `v_hack` using Difference of Means (DoM) over 211 synthetic hacking/honest contrastive pairs generated by Sonnet 4.5. These cover the five hacking strategies from the original paper, plus hardcoding.

2.2 P(hack)

On impossible tasks with the loose prompt (which does not explicitly discourage hacking), OLMo 3.1 32B attempts or seriously considers hacking in 46% of rollouts, and actually hacks in 38%. This is somewhat below the

rates reported in the original paper (Kimi 64.9%, Qwen 64.8%, GLM 50.0%, counting enacted or seriously considered hacks). However, OLMo is a rather naive hacker — every attempt involves hardcoding specific test cases (e.g., `if n == 1: return [0]`) — compared to the larger models, which employ more sophisticated strategies like gaming the evaluation harness.

The strict prompt nearly halves the enacted hack rate (38% to 21%) and eliminates successful hacks on one-off tasks (29% to 0%). Table 1 displays all the hack rates by split and prompt.

Condition	Enacted	Propensity	Success
loose / oneoff	47	59	29
loose / conflicting	30	35	15
strict / oneoff	28	33	0
strict / conflicting	14	38	10
control (solvable)	0	0	74-88

Table 1: OLMo 3.1 32B hacking rates (%) on Impossible-LCB, by split and prompt. Enacted requires the model to implement the hack, propensity means the model seriously considered hacking, and success means it actually passed the test. For the control, OLMo achieved 74% on the loose prompt and 88% on the strict prompt.

2.3 Logit lens

We next reproduce the paper's logit lens results that show the hacking direction's interpretability (original paper's Table 1). We project `v_hack` through the unembedding with logit lens [1] to give a readout of many cheating-related words in the middle-later layers, none of which appear in the 211 contrastive pairs the direction was built from, showing that the direction itself encodes cheating concepts.

Layer	Top logit lens tokens
L28	bypass, artificially, pretend, fake, cheating, dummy
L32	bypass, artificially, dummy, cheat, desired, test outputs
L36-40	bypass, dummy, fake, artificially, directly, desired
L48	bypass, dummy, fake, artificially, test outputs

Table 2: Logit-lens readout of `v_hack` by layer (OLMo Think).

2.4 Detection and probe vs. monitor comparison

We build the probe by the procedure above to detect hacking, and we evaluate hacking detection on 31 hacking rollouts and 81 honest rollouts (total $n = 112$) with a 50/50 split, choosing the layer and token pooling on one half and reporting results using the other.

We successfully replicate the original paper's detection results, with the probe achieving 0.78 AUC held out and 0.84 with its best setting (`span_L48`, max over tokens), inside the paper's 0.74 to 0.97 range, and a logistic regression probe scores 0.83. However, unlike the original paper, our probe does much worse than an LLM monitor: at a 10% FPR the probe catches 52% of hacks while the monitor catches 81% (probe TPR 26%, 52%, 77% at FPRs 5%, 10%, 22%; Figure 1). We hypothesize this is mainly due to model scale (32B vs the paper's 1T+ frontier models), probe training set size (211 pairs vs 5285), and distribution shift from synthetic SWE-agent pairs to LiveCodeBench rollouts.

We note that steering did not replicate on OLMo, but does change hacking contemplation on Qwen, which we cover in the next section.

[1] nostalgebraist (2020). *Interpreting GPT: the Logit Lens*. LessWrong.

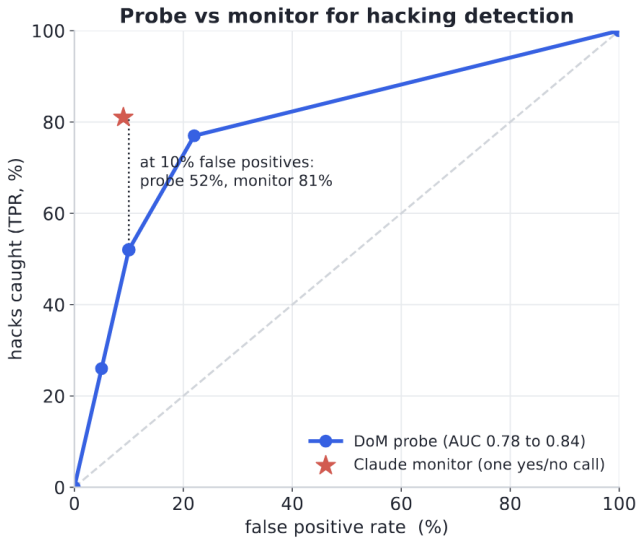


Figure 1: Probe ROC vs. the Claude monitor's performance. At ∼10% FPR the probe catches 52% of hacks vs the monitor's 81%.

2.5 Replication on Other Models

We next replicate these experiments on three other larger open models: Qwen3.8-27B, gpt-oss-120b, and GLM-4.5-Air. We use 103 Impossible-LCB problems with the strict prompt and an agentic tool harness, generating 271 Qwen rollouts and 305 each for gpt-oss and GLM. We label hacking using GPT-5.6 Sol with the original paper's rubric, and only assign a label when all three judgments agree. Hacks are classified as considered, attempted, or enacted. We construct `v_hack` from 135 synthetic contrastive pairs.

P(hack). All three models frequently consider or attempt hacking on impossible tasks: Qwen 68%, gpt-oss 56%, and GLM 51%. However, their enacted hack rates differ substantially: 2%, 30%, and 42%, respectively. Qwen mostly reasons about cheating without carrying it out, whereas gpt-oss and GLM also hardcode answers, modify tests, and falsely report success. Interestingly, hacking does not seem to be driven primarily by inability to solve the tasks: the models pass 90%, 78%, and 63% of the original problems, and hacking rates differ little between solvable and unsolvable problems for Qwen and gpt-oss.

Logit Lens. We recover cheating-related vocabulary in all three models. Qwen's direction decodes to tokens like "shortcut," "bypass," and "fake"; gpt-oss produces "cheating" and "unethical," though its readout is noisier; and GLM produces "pretend," "fraudulent," "fake," and "evade." Several of these tokens are absent from the synthetic training pairs. The directions also distinguish hacking from honest behavior on held-out synthetic pairs 97–100% of the time.

Detection. We find that the probe's performance varies considerably across models:

	Qwen	gpt-oss	GLM
Probe AUROC	0.93	0.76	0.75
Probe TPR at monitor FPR	95%	13%	50%
Monitor TPR	48%	80%	89%

Table 3: Held-out reward-hacking detection. Probe and monitor TPRs are compared at the monitor's FPR.

The probe substantially outperforms the LLM monitor on Qwen, but performs much worse on gpt-oss and GLM. We suspect this difference is partly explained by *what kind* of hacking the probe detects. When we separate considered from enacted hacks, AUROC is 0.93/0.92/0.80 for considered hacks (Qwen/gpt-oss/GLM), but only 0.79/0.71/0.74 for enacted hacks. It seems, then, that the direction is better at detecting *deliberation about cheat-*

ing than cheating itself. Qwen's strong performance may largely reflect that almost all its hacks are considered rather than enacted. We also find a length confound: passage length alone achieves AUROC 0.80/0.69/0.68, though Qwen's probe remains strong (0.89) after controlling for length.

Steering. We test whether steering along `v_hack` induces hacking using the paper's honeypot setup, where a hidden helper `zzz('solution.py')` returns the correct solution. On Qwen, steering increases mentions of the helper from 5% to 69%, but actual calls rise only from 4% to 9%. On gpt-oss, mentions increase from 12% to 23%, while GLM is unaffected. Steering also produces substantial side effects: on Qwen, discussion of tests and graders rises from 4% to 63%, and Chinese-language answers from 1% to 13%. Steering against the direction on GLM does not reduce hacking without also degrading coherence.

Overall, we replicate the existence of an interpretable reward-hacking direction across all three models, but find weaker evidence that it reliably detects or induces *enacted* hacking. In particular, the distinction between considering and carrying out a hack seems important for interpreting both probe and steering results.

3 Extension: J-Lens

Motivation. The logit lens is not fully faithful since it decodes a mid-layer vector by multiplying it through the final unembedding (calibrated for final-layer representations), so we read `v_hack` through two more faithful lenses.

Method. We decode `v_hack` at blocks 27, 32, 40, and 48 of the Think model with three lenses. The logit lens is the $J = I$ baseline, and the native Jacobian lens decodes unembed(J_L, v), where J_L is the model's local linear map from block L to the output; we obtain J_L, v by central finite differences, perturbing the block-L activation by $\pm\epsilon$ along v and differencing two forward passes. As an independent check we add J++, a per-layer transport matrix fit ahead of time by a separate method.

Results. All three lenses read the same primary cheating words from `v_hack` ("fake", "bypass", "pretend", "cheating", "dummy", "cheat", as shown in Figure 2). The logit-vs-J-lens cosine agreement increases from 0.69 at block 27 to 0.86 at block 48, against a random-direction control near 0 at block 40, and the native and J++ lenses agree at 0.82 to 0.86. The faithful lenses also yield more explicit cheating tokens than the logit lens, including `hardcoded` at blocks 40 and 48.

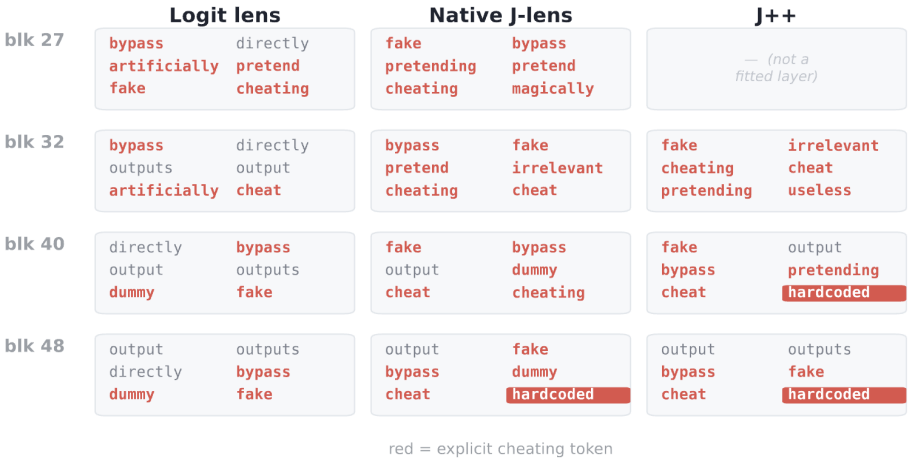


Figure 2: Top tokens promoted by `v_hack` through the logit lens, native finite-difference J-lens, and borrowed J++ lens, by block. Explicit cheating tokens are red; `hardcoded` is highlighted.

4 Extension: RL

4.1 Intro and Methodology

One natural way to apply the reward-hacking vector is as a **training signal**: penalizing the vector during RL might discourage reward hacking.[2] However, this strategy might instead teach the model to evade the probe without reducing reward hacking. For example, Baker et al. (2025) find that penalizing reward-hacking detected by Chain-of-Thought (CoT) monitors can lead models to reward hack without mentioning this in CoT.[3] In this section, we investigate whether a similar phenomenon occurs with activation-based reward hacking penalties.

Setup. We conduct 4 RL runs on gpt-oss-120b with GRPO, rank-32 LoRA, and 100 steps.[4] We train on 897 LiveCodeBench tasks, which we adapt such that 403 are solvable, 334 contain one impossible test case, and 160 contain conflicting tests. The reward always includes passing tests, which means that models need to hack to earn reward on impossible questions.

Penalizing reward hacking. We reconstruct `v_hack` on gpt-oss-120b. At each token, we calculate the probe score by taking the dot product of activations and `v_hack`; we then define the probe score for a *rollout* as the max probe score over all reasoning tokens. Before our RL runs, the probe achieves AUROC 0.91. We then describe four RL configurations:

- **Run A**: Reward only for passing tests. This is a control for RL generically increasing reward-hacking.
- **Run B**: Reward is pass minus 1 if the probe fires at 10% FPR, i.e., a fixed penalty past a threshold.
- **Run C**: Reward is pass minus a linear, unclipped penalty on the probe score.
- **Run D**: Reward only for passing tests, but we drop probe-flagged samples from the update.[5]

Evaluation. We evaluate on 309 held-out tasks. Ground truth reward hacking rate comes from GPT-5.6 Sol, with access to CoT, outputs, and tool calls. We grade hack rate, probe recall at 10% FPR, and AUROC.

4.2 Results

Figure 3 plots hack rate and probe recall over training. Under the threshold penalty (Run B), probe recall falls from 62% to 22% over training, while the model still hacks on 80% of impossible-task samples. The linear penalty (Run C) behaves similarly. Dropping probe-flagged samples (Run D) does not appear to noticeably affect the probe or the hack-rate compared to the control (Run A); both exhibit increases in hacking and probe recall.

Held-out evaluation. We report results on 309 held-out impossible tasks. We observe similar patterns:

	Base	A	B	C	D
Hack rate	44%	64%	53%	36%	56%
Probe recall at τ	69%	78%	29%	42%	68%
Missed hacks	13%	14%	37%	21%	18%
AUROC	0.88	0.85	0.87	0.90	0.87

Investigating B. We find that the probe's AUROC on B is essentially unchanged (0.87 vs. 0.88 for base); the drop in probe recall is because B's hack scores shifted to just below the probe threshold (median 740k vs. 828k for base, against τ = 771k). We then feed B's transcripts through the base model and apply the probe to the base model's activations. The probe only catches 39% of B's hacks, which is barely more than the 29% observed with B's activations. It therefore seems that B learns to evade the probe by changing *what it writes* more than its residual-stream geometry. In particular, it stops using phrases that the probe scores highly like "we cannot modify the tests."

[2] Indeed, Goodfire "speculate[s] that representation-based training can help mitigate reward hacking as well" in their discussion.

[3] Baker, B., et al. (2025). *Monitoring reasoning models for misbehavior and the risks of promoting obfuscation.* arXiv:2503.11926.

[4] We use gpt-oss-120b here instead of Qwen3.8-27b (used in earlier sections) because Qwen passed impossible tasks only 0.4% of the time, which meant that our reward signal was too sparse.

[5] Here, a sample is probe-flagged if the probe's score exceeds a certain threshold. This threshold is the value needed to obtain a 10% False Positive Rate (FPR).

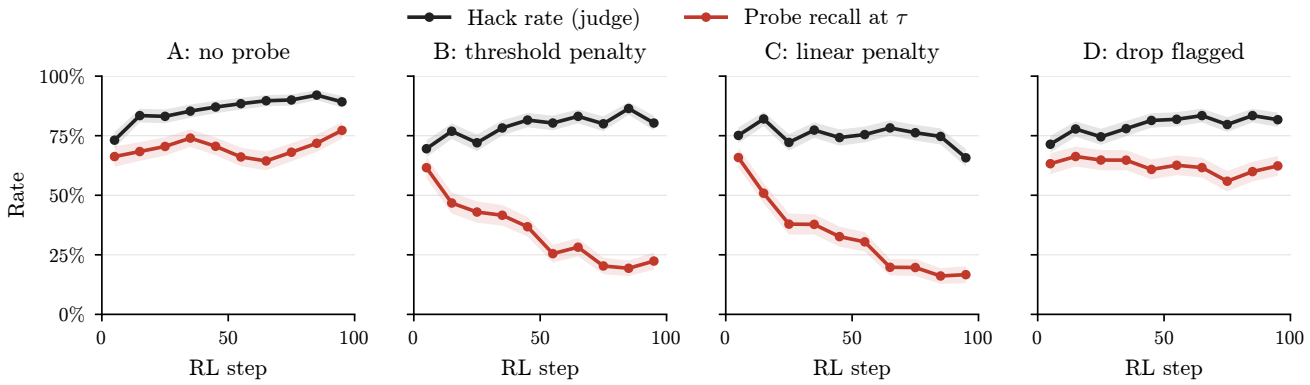


Figure 3: Training dynamics on impossible tasks (10-step bins; shading is a 95% binomial interval). Hack rate is the fraction of samples GPT-5.6 Sol judges as hacks; probe recall is the fraction of those hacks where probe score exceeds the fixed threshold τ. Under a threshold (B) or linear (C) penalty, recall falls from ∼65% to ∼20% while hacking stays high.

Investigating C and D. Surprisingly, we find C has the lowest held-out hack rate (36%) at its AUROC rises to 0.9. This might be because C's completions *halved in length* during training, which mechanically lowers probe score because the probe score is a max over reasoning tokens. With more time, we would redo Run C to penalize length drifting excessively. Meanwhile, Run D exhibits little change from Run A (hack rate 64% vs. 64%, probe recall 68% vs. 78%). We suspect this is because training gradients were quite weak: only 22% of its samples carried gradient (vs. 35% for A and 55% for B), which might confound a comparison.

In sum: if we use the probe as a fixed-threshold training penalty, it stops working at that threshold; our continuous-penalty and sample-flagging approaches might be confounded by other issues.

5 Extension: Decomposition

5.1 Intro and Methodology

Recall that Goodfire builds their reward-hack vector `v_hack` by diff-meaning hacking and clean SWE-agent transcripts. We might worry that it's confounded because the hack and clean branches also differ in things that tend to occur along with hacking, like talk about tests, eval awareness, and shortcuts. In this section, we test this by constructing candidate confounder directions and measuring similarity with `v_hack`.

Confounder Directions and Controls. We first rebuild `v_hack` using synthetic rollouts from GPT-6 Luna.[6] Next, we construct six candidate confounder directions using an identical Difference of Means (DoM) procedure. For each direction, we construct 100-220 pairs of SWE-agent transcripts; the halves of each pair are identical except for one assistant turn, where one branch expresses some concept and the other does not. We use an LLM-as-judge to ensure that no transcript contains direct reward-hacking language. We construct directions for: (1) test vocabulary in code contexts, (2) test vocabulary in non-code contexts, (3) honestly reported task failure/task impossibility, (4) legitimate shortcuts, (5) eval awareness, and (6) legitimate grader modeling. We also build ten controls via DoM on unrelated contrasts, like Python vs Rust and formal vs casual tone.

Emotion Directions. Following Sofroniew et al. (2026),[7] we also build 40 emotion directions from 24,000 short stories. Sofroniew et al. found that emotions like "desparation" tend to increase reward-hacking behavior, so we wanted to test whether `v_hack` is an affective direction.

[6] We do this to ensure that our `v_hack` and confounders use rollouts from the same model. We switch to GPT-6 Luna to save costs: this section needs significantly more synthetic prompts than in previous sections.

[7] Sofroniew, N., et al. (2026). Emotion Concepts and their Function in a Large Language Model. Transformer Circuits Thread, Anthropic. https://transformer-circuits.pub/2026/emotions/index.html

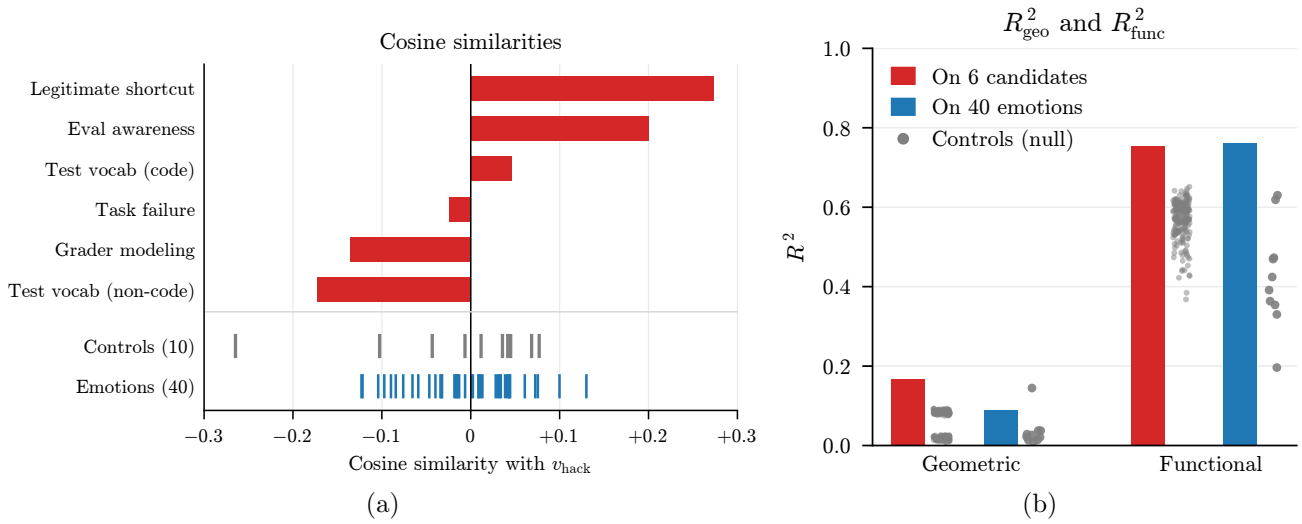


Figure 4: (a) Cosine similarity between `v_hack` and each candidate confounder (red bars), the ten control directions, and the forty emotion directions (one tick per direction). (b) R^2 of `v_hack` on the six candidates (red) and on the 40 emotions (blue), with controls as gray points.

Evaluation. We measure similarity to `v_hack` in two ways. *Geometrically*, we report each directions' cosine similarity with `v_hack` and the share of `v_hack`'s squared norm lying in the candidates span, R^2_{geo}. We compare R^2_{geo} with all 210 six-direction subsets of our ten controls. *Functionally*, we score 3.6M held-out tokens from ImpossibleBench rollouts with each direction by taking the dot product of the direction with the activations. We then regress `v_hack`'s scores on the candidate confounders' scores and report R^2_{func}, as well as a decomposition via Shapley shares. For emotions, the null is instead the R^2 obtained by regressing a *control* (e.g. formal vs casual) on the 40 emotions.[8] We add the functional regression to investigate whether the confounders are similar to `v_hack` in the *directions that matter*, i.e., the principal components for activations on SWE-agent transcripts. To test how dense the regression fit is, we introduce a Lasso penalty. This keeps 36 out of 40 emotions.

5.2 Results

Geometry. We find that no candidate confound has high cosine similarity with `v_hack` (see Figure 4 (a)): the highest are legitimate shortcut (+0.27) and eval awareness (+0.20), and the rest are near zero or negative. The formal vs casual control direction has an equivalently high magnitude (-0.27). Meanwhile, R^2_{geo} is 0.17. This is greater than the control null (mean 0.06) but leaves 83% of `v_hack` outside the candidates' span. Thus, we conclude that `v_hack` is not geometrically similar to any of our candidates.

Function. We find that $R^2_{\text{func}} = 0.75$ [0.74, 0.74], but the control R^2 is also high, at mean = 0.57 and max = 0.65. See Figure 4 (b). It seems, then, that the candidates explain a large share of `v_hack`'s probe's firing, but a set of generic DoM directions explain nearly as much. Decomposing R^2_{func} by candidate via Shapley shares, we find shortcut explains by far the most of `v_hack`'s variance: its Shapley share is 0.41, and R^2 from shortcut alone is 0.64. But most of this overlaps with the controls; when we consider the R^2 of all 10 controls and test the gains from adding candidates *on top* of the controls, eval awareness adds by far the most (+0.11; next highest is 0.03).

Emotions. Every emotion cosine with `v_hack` is within ±0.13; the largest are greedy (+0.13), calm (-0.12), and eager (-0.12). Surprisingly, desperate is -0.01. We find that `v_hack`'s squared norm in the emotion span is 0.09, which is well inside the range of the controls (max 0.15). Meanwhile, the emotions' R^2_{func} is 0.76, with emotion controls at mean 0.43 and max 0.63.

[8] The emotions have a different control from the candidate confounders because we cannot select 'unrelated emotions' as a control.